

PRD: "BitForge" Open-Source Protocol Emulator ASIC (Verilog + UVM Edition)

Version: ~~0.2 (draft)~~ 0.3 (draft, Verilator 5.052 update) | **Date:** 2026-10-04 | **Target:** Tiny Tapeout IHP CMOS5L (130 nm), 6x4 tiles | **Deadline:** 2027-01-18 | **Shuttle:** March 2027

Differences from v0.1: the DUT is hand-written synthesizable **Verilog-2005** (no Hardcaml, no generators). Verification uses **SystemVerilog** ~~UVM 1.2 (IEEE 1800.2)~~ UVM (IEEE 1800.2-2017/2020) simulated primarily with Verilator 5.052 with SVA assertions. Requirement IDs are <AREA>-nnn . Priority: **M** Must, **S** Should, **C** Could.

Redline legend (v0.3): ~~red strikethrough~~ = v0.2 text that was revised; green underlined = replacement or added text. All v0.2 text is retained.

1. Overview

1.1 Problem

Bit-banging protocols from a general CPU is timing-fragile; fixed peripherals are inflexible. Hardware debuggers and reverse engineers need a small, deterministic chip that can speak, sniff, and fuzz arbitrary wire protocols, including ones defined after fabrication.

1.2 Vision

A cycle-deterministic, multi-lane programmable I/O processor. Each lane is a minimal engine whose ISA is built around pin I/O, cycle counting, and bit shifting. Protocols are firmware, not silicon.

1.3 Goals

1. UART, SPI and I2C implemented purely in firmware.
2. New protocols possible post-fabrication within documented timing/I/O limits.
3. Fit 24 tiles (~24K cells) and close timing after full place and route.
4. Readable, reviewable Verilog RTL that is verified by a reusable UVM environment.
5. Fully open source with a reproducible flow.

1.4 Non-goals

Analog PHYs (USB/Ethernet transceivers, magnetics), protocols above ~100 Mbit/s, general-purpose compute.

1.5 Personas

ID	Persona	Need

P1	Hardware debugger	Sniff and decode buses, trigger on patterns
P2	Reverse engineer	Talk to or fuzz unknown serial interfaces
P3	Firmware author	Write, simulate and load protocol firmware
P4	Verification engineer	Reuse UVM agents, sequences and coverage

2. Platform Constraints

ID	Constraint	Pri
CON-001	Process: IHP 130 nm CMOS5L through Tiny Tapeout, using the <code>cmos5l</code> Verilog template	M
CON-002	<code>info.yaml</code> tile size 6x4; design must also build unchanged if 8x4 becomes available	M
CON-003	Interface limited to the Tiny Tapeout wrapper: <code>ui_in[7:0]</code> , <code>uo_out[7:0]</code> , <code>uio_in/out/oe[7:0]</code> , <code>clk</code> , <code>rst_n</code> , <code>ena</code>	M
CON-004	Budget ~1K cells/tile; leave headroom for clock tree and routing	M
CON-005	All design and verification source open source (Apache-2.0 RTL; verification code Apache-2.0)	M
CON-006	Submission by 2027-01-18	M
CON-007	DUT RTL is plain Verilog-2005 only, synthesizable by Yosys/OpenROAD flow; SystemVerilog is permitted only in the testbench, bind files and assertion modules, never in synthesized RTL	M

3. Architecture Requirements

Reference: **N=4 lanes** sharing instruction memory, a pin crossbar, FIFOs and a host-control block.

ID	Requirement	Pri
ARC-001	4 independent lanes (Verilog <code>parameter NUM_LANES</code> , 2-8), each with PC, registers, shifter and cycle counter	M
ARC-002	Shared instruction memory, at least 256 x 16-bit (target 512), with per-lane base and wrap addresses	M
ARC-003	Instruction memory loadable at runtime via the host interface; each lane can be stopped, reprogrammed and restarted independently	M
ARC-004	Single clock domain; no gated or derived clocks	M
ARC-005	Global "sync start" launches selected lanes on the same clock edge	M
ARC-006	Instruction memory wrapped in one module (<code>bf_imem</code>) with interchangeable	S

	implementations (Flip-Flop, latch, SRAM macro) selected by <code>define/parameter</code>	
ARC-007	Per-lane TX and RX FIFOs, minimum 4 x 32-bit each, with level/empty/full flags	M
ARC-008	32-bit free-running timestamp counter, readable by lanes and host, latched on pin-edge events	S
ARC-009	8 shared event flags usable for lane-to-lane coordination (set, clear, wait)	M
ARC-010	Reset puts all lanes in a halted state with defined register values; GPIOs tristate	M
ARC-011	RTL partitioned into small documented modules with a module-level spec each: <code>bf_lane</code> , <code>bf_alu</code> , <code>bf_shift</code> , <code>bf_fifo</code> , <code>bf_xbar</code> , <code>bf_imem</code> , <code>bf_hif</code> , <code>bf_regs</code> , <code>bf_ts</code> , <code>bf_top</code>	M

4. Instruction Set Architecture

Design principle: every instruction is **single-cycle and deterministic**, so timing equals instruction count.

ID	Requirement	Pri
ISA-001	Fixed 16-bit instructions	M
ISA-002	Non-stalling instructions execute in exactly 1 cycle; stall conditions documented cycle by cycle	M
ISA-003	Per-lane registers: 4 x 16-bit general (X, Y, Z, W), 32-bit input shift register (ISR), 32-bit output shift register (OSR)	M
ISA-004	Pin instructions: <code>IN</code> , <code>OUT</code> , <code>SET</code> , <code>DIR</code>	M
ISA-005	Side-set: 1-3 configurable bits per instruction update pins in the same cycle as the main operation	M
ISA-006	<code>WAIT</code> on pin level/edge, event flag or timestamp, with optional timeout	M
ISA-007	<code>JMP</code> conditions: always, <code>reg==0</code> , <code>reg!=0</code> (post-decrement), pin state, shift-count threshold, FIFO state, flag	M
ISA-008	3-bit <code>DELAY</code> field (0-7 cycles) on every instruction	M
ISA-009	<code>SHIFT</code> with direction, count 1-32, auto-push/auto-pull at configurable threshold	M
ISA-010	<code>MOV</code> , <code>ADD/SUB</code> (immediate), <code>AND/OR/XOR/NOT</code> , bit-reverse, compare	M
ISA-011	<code>CALL</code> / <code>RET</code> with a 2-deep return stack	S
ISA-012	Optional majority-vote oversampling on inputs (3-of-5)	S
ISA-013	Host IRQ instruction raising a status bit and optional pin	M
ISA-014	NRZI and bit-stuffing mode flags (for USB LS, CAN)	C
ISA-015	Manchester encode/decode helper (for 10BASE-T)	C

ISA-016	Optional CRC unit (polynomial and width configurable, up to 16 bits plus CRC-32 mode)	S
ISA-017	Illegal opcode halts the lane and sets a sticky fault flag	M
ISA-018	ISA specified in a single document and a machine-readable table (<code>isa.yaml</code>) used by the assembler, ISS and UVM instruction-generator	M

5. I/O Requirements

ID	Requirement	Pri
IO-001	Up to 16 lane-controllable pins (<code>uio</code> , <code>uo_out</code>); <code>ui_in</code> sample-only except pins reserved for the host channel	M
IO-002	Crossbar register assigns each pin's output to a lane; any lane can read any pin	M
IO-003	Multiple lanes on one pin: lowest lane index wins, sticky conflict flag set	M
IO-004	Open-drain emulation per pin (output enable toggled, data held low), for I2C	M
IO-005	2-flop input synchronizer per pin with bypass mode; latency in cycles documented for both	M
IO-006	Optional input glitch filter (1-4 cycles) per pin	S
IO-007	Per-pin input and output inversion	S
IO-008	All pins tristate in reset	M

6. Timing and Performance

ID	Requirement	Pri
TIM-001	System clock ≥ 50 MHz post-route (stretch 100 MHz); achieved f_{max} published	M
TIM-002	One pin transition per lane per clock (≥ 25 MHz square wave at 50 MHz)	M
TIM-003	Pin-edge to pin-response latency ≤ 4 clocks in bypass mode	M
TIM-004	Per-lane 8.8 fractional clock divider; 115200 baud within 1% from 50 MHz	M
TIM-005	Zero cycle skew between lanes started by sync start	M
TIM-006	Published table of maximum bit rate per protocol	M

7. Host Interface

ID	Requirement	Pri
----	-------------	-----

HIF-001	SPI slave (SCK, CS, MOSI, MISO, 4 pins) for control	M
HIF-002	Memory-mapped register space: program memory, lane config, FIFOs, flags, crossbar, status, timestamp	M
HIF-003	Burst write for program memory and burst read/write for FIFOs in one transaction	M
HIF-004	Interrupt/attention output pin	M
HIF-005	Strap-selectable UART boot path	C
HIF-006	Chip ID and version registers	M
HIF-007	Debug visibility: per-lane PC, registers, halt, single-step	S
HIF-008	PC breakpoint per lane	C
HIF-009	Host SPI clock up to clk/4	M
HIF-010	Register map defined once in a single spec file (<code>regmap.yaml</code>) and used to generate the UVM RAL model (see UVM-011)	M

8. Protocol Support

ID	Protocol	Role	Pri
PRO-001	UART, 5-8 bits, parity, 1-2 stop, 300 baud to ≥ 3 Mbaud	TX, RX, sniff	M
PRO-002	SPI modes 0-3, up to clk/4, word size 1-32	Master, slave, sniff	M
PRO-003	I2C 100k/400k/1M, clock stretching	Master, slave, sniff	M
PRO-004	JTAG (TAP navigation, shift IR/DR)	Master	S
PRO-005	SWD (request/ack/data with turnaround)	Master	S
PRO-006	PS/2	Host, device	S
PRO-007	CAN 2.0 at ≤ 500 kbit/s	Sniff, then TX	C
PRO-008	USB low-speed 1.5 Mbit/s (NRZI, stuffing, CRC-16), external analog front end	Device-level handshake	C
PRO-009	10BASE-T Ethernet (Manchester, link pulse, TX) via external PHY	TX / loopback	C
PRO-010	1-Wire, WS2812, DShot, DMX, I2S	Master	C
PRO-011	Multi-lane composition demo (clock lane + data lane)		S

9. Differentiating Capabilities

ID	Feature	Pri
DIF-001	Triggered, timestamped, run-length-encoded edge capture (logic analyzer mode)	S
DIF-002	Auto-baud measurement	S
DIF-003	LFSR source for glitch/fuzz generation with deterministic seed	S
DIF-004	Man-in-the-middle bridge mode using two lanes	C
DIF-005	Pattern-match trigger output pin	S
DIF-006	Reference firmware library, versioned	M
DIF-007	Cycle-accurate ISS used as the UVM reference model, bit-for-bit with RTL	M

10. Software Tooling

ID	Requirement	Pri
SW-001	Assembler with labels, macros, side-set syntax and per-line cycle annotation	M
SW-002	Disassembler	S
SW-003	Cycle-accurate ISS with VCD pin waveform output; also built as a DPI-C library for the UVM scoreboard	M
SW-004	Python host library for loading, config and FIFO access via an RP2040 or FT232H bridge	M
SW-005	Firmware static timing analyzer	C
SW-006	Example firmware per PRO item	M

11. RTL Design Requirements (Verilog)

ID	Requirement	Pri
RTL-001	Verilog-2005, synthesizable subset; one module per file, file name equals module name	M
RTL-002	Coding style guide published: synchronous active-low reset (<code>rst_n</code>), non-blocking assignments in sequential logic, no inferred latches in combinational logic, explicit <code>default</code> in <code>case</code> , no <code>initial</code> blocks in RTL	M
RTL-003	All parameters and <code>localparam</code> constants (opcodes, register offsets) in a single header <code>bf_defs.vh</code>	M
RTL-004	Single clock domain; all asynchronous inputs through synchronizers (IO-005)	M
RTL-005	Zero unwaived Verilator <code>--lint-only -Wall</code> and Yosys warnings	M
RTL-006	Build profiles via defines: <code>BF_PROFILE_MIN</code> , <code>BF_PROFILE_STD</code> , <code>BF_PROFILE_MAX</code>	M

RTL-007	Assertions and coverage live in separate bind ed SystemVerilog modules, so synthesized RTL stays pure Verilog	M
RTL-008	Every module has a header comment: purpose, ports, timing/latency, reset behavior	S
RTL-009	FPGA prototype (any open-toolchain board) runs PRO-001..003 against real devices	S

12. Verification Requirements (UVM)

12.1 Strategy and environment

ID	Requirement	Pri
UVM-001	UVM 1.2 testbench <u>UVM testbench (IEEE 1800.2-2017 or 2020-3.2, whichever Verilator 5.052 runs cleanly)</u> in SystemVerilog, top-level <code>tb_top</code> instantiating the DUT through the Tiny Tapeout wrapper ports, <u>compiled and run with Verilator 5.052 (--timing , --coverage , --binary / --build)</u>	M
UVM-002	A reusable <code>bf_env</code> composed of agents, scoreboard, reference model, coverage collector and virtual sequencer	M
UVM-003	Interfaces: SV <code>interface</code> for host SPI, one for the 24-pin GPIO bus, with modports and clocking blocks; DUT attached through virtual interfaces in <code>uvm_config_db</code> ; <u>the GPIO bus is modeled as separate <code>in / out / oe</code> signals (no <code>inout</code> nets) and I2C wired-AND is resolved in the interface/agent, because Verilator handles tri-state poorly</u>	M
UVM-004	Testbench is clock/reset aware with a dedicated <code>reset_agent</code> or reset sequence supporting mid-test reset	M
UVM-005	All components factory-registered and configurable by <code>uvm_object</code> config classes; no hard-coded constants in components	M
UVM-006	Log/verbosity discipline: <code>uvm_info/warning/error/fatal</code> with defined ID naming; a test fails on any <code>UVM_ERROR / UVM_FATAL</code>	M

12.2 Agents

ID	Agent	Behavior	Pri
UVM-007	<code>spi_host_agent</code> (active)	Drives HIF-001 transactions; item with opcode, address, burst data; monitor reconstructs transactions	M
UVM-008	<code>pin_agent</code> (active/passive per pin group)	Drives <code>ui_in / uio_in</code> with programmable timing, glitch and jitter injection; monitor records edge-timestamped pin activity from <code>uo_out / uio_out/oe</code>	M
UVM-009	<code>uart_agent</code>	Protocol-compliant partner (TX/RX, configurable baud/parity/stop, error	M

		injection: framing, parity, break)	
UVM-010	spi_dev_agent	SPI master or slave partner, modes 0-3, configurable word size	M
UVM-011	i2c_agent	I2C master/slave partner with clock stretching, ACK/NACK, arbitration loss injection	M
UVM-012	jtag_agent , swd_agent , ps2_agent	Partner models for the S-priority protocols	S
UVM-013	can_agent , usb_ls_agent , eth10_agent	Partner models for stretch protocols	C
UVM-014	Every protocol agent includes a protocol-checker (monitor-side) that flags spec violations independently of the DUT	M	

12.3 Register model, reference model and scoreboards

ID	Requirement	Pri
UVM-015	RAL (uvm_reg_block) generated from regmap.yaml , with adapter and predictor for the SPI host agent; built-in register tests (reset values, bit-bash, access) run	M
UVM-016	Reference model: ISS (SW-003) bound via DPI-C , stepping in lockstep with the DUT using the DUT's start/stop/sync events; DPI-C operation under Verilator is proven in UVM-047	M
UVM-017	lane_scoreboard : compares per-cycle pin output, PC trace, and FIFO traffic against the reference model; mismatches report cycle and lane	M
UVM-018	protocol_scoreboard : compares bytes/words observed by partner agents against the firmware's expected data, per protocol	M
UVM-019	fifo_scoreboard : ordering, level and flag correctness for all FIFOs	M
UVM-020	Timing checker: measured edge-to-edge intervals are compared against ISA-defined cycle counts with zero tolerance for ISA-level timing and a configurable tolerance for protocol-level timing	M

12.4 Sequences and tests

ID	Requirement	Pri
UVM-021	Layered sequences: base item sequences, host-config sequences (load program, set crossbar, start lanes), protocol scenario sequences, virtual sequences coordinating host and pins	M
UVM-022	Constrained-random instruction generator : legal instruction streams from isa.yaml , constraints for loop bounds, valid jumps, and weighted opcode mix, with	M

	directed bias options to hit corner cases; randomization runs through Verilator's constraint solver, which needs Z3 installed at run time (UVM-048)	
UVM-023	Directed tests: every opcode, every JMP condition, delay field extremes, side-set combos, auto-push/pull thresholds, stalls	M
UVM-024	Protocol tests: PRO-001..003 in all roles, sweeping baud/mode/speed, error injection, back-to-back and long-run	M
UVM-025	Host-interface stress: random burst lengths, back-to-back transactions, CS glitches, illegal addresses	M
UVM-026	Multi-lane tests: sync start, flag handshakes, pin conflicts, simultaneous FIFO access	M
UVM-027	Reset tests: reset during run, during burst, during FIFO traffic; state must return to known values	M
UVM-028	Illegal-opcode and fault-handling tests (ISA-017)	M
UVM-029	Regression suite with named test lists: smoke (< 5 min), nightly , full ; each test seed logged and replayable	M
UVM-030	Tests for S/C protocols (JTAG, SWD, PS/2, CAN, USB LS, Ethernet)	S

12.5 Coverage, assertions, formal

ID	Requirement	Pri
UVM-031	Functional coverage model (covergroup) covering opcode x operand classes, JMP conditions x outcomes, shifter states, FIFO levels x events, pin events, crossbar configs, lane-pair interactions, register accesses; built with Verilator --coverage , with cross-coverage support confirmed in UVM-047 (fallback: manual cross counters in the scoreboard)	M
UVM-032	Cross coverage of protocol parameters (baud/mode/speed x error type x role)	M
UVM-033	Functional coverage goal >= 95%, code coverage (line, branch, toggle, FSM) >= 90%, each exclusion reviewed and documented	M
UVM-034	SVA assertion modules bound to the DUT: FIFO overflow/underflow, FSM legal state, reset values, tristate in reset, pin-conflict flag, stall cycle counts, HIF handshake rules; concurrent-assertion and bind support in Verilator 5.052 is confirmed in UVM-047 (fallback: immediate assertions in a checker module instantiated in tb_top)	M
UVM-035	Assertions double as coverage (cover property) and are reported in the regression dashboard	S
UVM-036	Formal property checking on the SVA properties with SymbiYosys (or other available tool) for FIFO, HIF FSM and lane FSM	S
UVM-037	Formal sequential equivalence of the pre- and post-synthesis netlist where tools permit	C

12.6 Gate-level, tools and process

ID	Requirement	Pri
UVM-038	The same UVM environment runs unchanged against the post-synthesis netlist and post-route netlist (with SDF back-annotation if available) <u>(functional zero-delay simulation; SDF back-annotation is not assumed available in Verilator, so gate-level timing is covered by static timing analysis, PHY-003)</u> ; reduced test list <code>gl_s_smoke</code> required to pass	M
UVM-039	Simulator support matrix documented. Primary: a UVM-capable simulator available to the team (Questa, VCS, Xcelium, or Riviera-PRO). Secondary: Verilator with UVM support where available. <u>Primary: Verilator 5.052 (UVM 2020-3.2 support, with limitations tracked upstream in issue #1538). Fallback: a commercial UVM simulator if available to the team (Questa, VCS, Xcelium, or Riviera-PRO), or a non-UVM SV/cocotb testbench built around the same agents.</u> RTL-only tests must also run with Icarus/Verilator using a non-UVM wrapper (cocotb or plain SV) for community use	M
UVM-040	CI runs lint, <code>smoke</code> regression, and coverage merge on every commit; <code>nightly</code> on schedule; <u>the CI image pins Verilator 5.052 and Z3</u>	M
UVM-041	Coverage database merged across seeds; a closure report is generated and archived with each release	M
UVM-042	AI-assisted verification: LLMs may draft sequences, assertions, covergroups and bug triage; every artifact is human-reviewed and tagged <code>// AI-assisted</code> in the source with the prompt summary in the repo	S
UVM-043	Bug tracking: every RTL bug found by verification gets an issue with a reproducing test and seed, and a regression test added	M
UVM-044	Post-silicon bring-up tests: self-test firmware, loopback checks, and expected waveforms derived from UVM tests	M
UVM-045	Verification plan (vPlan) document mapping each requirement ID in this PRD to tests, assertions and coverage items, kept up to date and checked at release	M
UVM-046	Mutation sampling: inject at least 20 RTL mutants (inverted conditions, off-by-one cycle counts) and report the kill rate	C
UVM-047	<u>Verilator UVM spike before the M0 gate (about 6-8 hours): one small UVM test must demonstrate a virtual-interface agent, a scoreboard, <code>randomize()</code> with <code>{}</code> under Z3, a covergroup with a cross, a <code>uvm_reg_block</code> with predictor, a DPI-C call into the C ISS, and a bound SVA module. Pass/fail decides whether Verilator is the primary simulator</u>	<u>M</u>
UVM-048	<u>Z3 (or a compatible SMT solver) is installed on development and CI hosts for constrained randomization; solver choice documented (<code>VERILATOR_SOLVER</code>)</u>	<u>M</u>
UVM-049	<u>Verilator version (5.052) and UVM library version are pinned in the repo (container or lockfile); upgrades only after the full regression passes</u>	<u>S</u>

13. Physical Design

ID	Requirement	Pri
PHY-001	Post-synthesis area <= 65% of nominal 6x4 area at STD profile	M
PHY-002	Full Tiny Tapeout flow completes without DRC/LVS errors	M
PHY-003	Setup and hold met at target clock across supported corners	M
PHY-004	Synthesis area/fmax tracked in CI from week 2	M
PHY-005	Power estimate documented	S
PHY-006	MIN profile fallback that fits comfortably	M

14. Documentation and Open Source

ID	Requirement	Pri
OSS-001	Public repo with permissive license from day one	M
OSS-002	Datasheet: pinout, registers, ISA, timing, limits	M
OSS-003	Tutorials: "UART in 10 instructions", "add a new protocol"	S
OSS-004	One-command build to GDS; one-command regression launcher (make smoke , make nightly)	M
OSS-005	Dev-board and analog front-end notes	S
OSS-006	Verification user guide: how to write a new agent/sequence for a new protocol	S
OSS-007	Contribution guide and issue templates	S

15. Repository Structure (informative)

```
rtl/ (pure Verilog) | rtl/bf_defs.vh | veril/uvm/{agents,env,seq,tests,ral,cov} |  
verif/sva/ | veril/formal/ | model/iss/ (C + DPI) | sw/{asm,host,fw} | spec/  
{isa.yaml,regmap.yaml,vplan} | flow/ (Tiny Tapeout info.yaml , constraints) | docs/
```

16. Milestones

Milestone	Date	Exit criteria
M0	2026-10-18	ISA and regmap v0.1 frozen; repo, CI, flow template running; vPlan skeleton; Verilator UVM spike passed (UVM-047)
M1	2026-11-01	Single-lane Verilog + assembler + ISS; UVM env with SPI host + pin agents;

		UART passes
M2	2026-11-22	4 lanes, crossbar, FIFOs, HIF; RAL tests pass; SPI and I2C agents/tests; first area/timing report
M3	2026-12-13	Scoreboard with DPI reference model; random instruction regression; coverage > 85%; assertions + formal; FPGA demo
M4	2027-01-04	Coverage closure, GLS pass, P&R closure, S-level protocols
M5	2027-01-18	Submission: docs, datasheet, final GDS, vPlan sign-off, release tag

17. Risks

ID	Risk	Mitigation
RSK-001	Instruction memory area overrun	ARC-006 swappable memory; MIN profile; early synthesis
RSK-002	Timing closure below 50 MHz	Shallow single-cycle datapath; pipeline decode if needed
RSK-003	Pin count limits protocols	Flexible crossbar; document pin budgets
RSK-004	USB/Ethernet analog needs	Keep stretch; define external front end
RSK-005	UVM requires a commercial or partially supported simulator RSK-005	UVM depends on Verilator 5.052's partial UVM support (known limitations in upstream issue #1538) , which community contributors may lack
RSK-006	DPI-C integration differs per simulator	Isolate DPI behind a thin API; test on at least two simulators
RSK-007	Verification scope exceeds schedule	Prioritize M items; vPlan drives ordering; reuse agents across protocols
RSK-008	Hand-written Verilog has no generator to enforce consistency across ISA, assembler and docs	ISA-018 and HIF-010: shared YAML specs plus a CI consistency check that decode tables in RTL match <code>isa.yaml</code>
RSK-009	Tile size changes (8x4)	CON-002; build profiles scale
RSK-010	Verilator UVM features change or regress between releases	UVM-049: pin versions; upgrade only after regression passes

18. Success Criteria

1. **SC-1:** PRO-001..003 pass in RTL, gate-level and FPGA.
2. **SC-2:** At least three of PRO-004..007 demonstrated.
3. **SC-3:** GDS flow clean, timing met, area per PHY-001.

4. **SC-4:** UVM regression passes with functional coverage $\geq 95\%$, code coverage $\geq 90\%$, vPlan